可审计性是一种产品能力

24/24 与 0.0714：一个时序分析 Agent 的可信度工程

qiaotong555 · mtsa-agent 项目札记

两个数字同时为真。

第一个：项目自建的独立结论门禁，**24 项全部通过**——趋势、季节性、异常点、异常事件、变化点、需求覆盖、证据覆盖，一项不缺。

第二个：同一套代码，在 10 份真实气象数据上，对 14 条 NOAA 官方灾害事件的事件级召回是 **1/14 = 0.0714**。

只看第一个数字，这套东西“能做气象灾害分析”；加上第二个，它只能说“能检出统计意义上的异常，检不出人定义的灾害事件”。

这篇文章讲的是：怎么让这两个数字同时出现在同一份报告里。

一、“跑通”是个很低的门槛

项目最初级的验收很朴素：10 份真实数据 × 4 个问题 × pipeline/agent 两种模式 = 80 次运行，80 次成功，每次都有报告，证据覆盖率 1.000。

四个指标全绿。但“成功”在这里只意味着：进程没崩、产物落了盘、结论数不为零。**它不表示结论是对的。**

真正让我警惕的是一个更细的观察：如果把这套系统交给一个不懂统计的人，他看到的是一屏格式整齐、措辞自信的结论——平均每个问题 39 条。他没有任何办法分辨哪条是“数据撑得住的”，哪条是算法在噪声里找到的图案。

所以后来我把“可信”拆成三件可以验收的事：

1. **每条结论能不能被追问**到具体数字、具体文件、具体计算位置；
2. **阈值是不是在跑之前就写死的**；

3. **没做的分析项有没有写清“为什么没做”**。

这三件事都不性感，但它们决定了报告是“参考”还是“证据”。

二、让每条结论都能被追问

一条结论不是一句话，是一个结构：结论文本 + 证据列表 + 来源工具。

比如“温度存在上升趋势”这句话，在系统里必须同时带上：趋势斜率、p 值、参与计算的时间区间、产出这些数字的产物文件，以及“第几个节点算的”。缺任何一项，这条结论就不许进报告。

这套约束带来的第一个副作用是：**分析代码必须知道自己算的是什么**。于是我不得不把“12 / 13 / 14 项”这三个数字分清楚——规则计划实际选中 12 个分析项；计划层的工具规格表有 13 项（多一个 `randomness`）；引擎内部的开关有 14 项（再多一个 `volatility`）。评估里说的“规则计划覆盖率”永远以 12 为分母，“工具覆盖率”永远以 5 个白名单工具为分母。

这两个分母混用一次，覆盖率就会“变好”一次。工程上这叫口径，产品上这叫骗人。

第二个副作用更重要：**未执行也必须是一个可报告的状态**。

系统的每个工具只能处于三种状态：`ok`、`failed`、`not_run`。而 `not_run` 必须附带原因，分四类：

- 模型没选它（`model_skipped`）；
- 被范围策略拒绝（`scope_exceeded`，追加步数用尽）；
- 运行被提前终止（预算耗尽 / 步数上限）；
- 前置条件不满足。

为什么值得为这件事写代码？因为它直接决定了另一件事能不能被解释——下一节。

三、先把阈值写死，再看结果

第二份数据集（`weather_eval_v2`）是确定性生成的：2 个站点、5 列、一年小时级，异常按已知位置注入。在跑之前，24 项检查的阈值全部写进任务书并冻结，跑完不允许改。

结果 24/24，异常事件召回 0.9667，变化点 F1 0.50（阈值 0.50，压线）。

如果文章到这里结束，它是一篇成功案例。但压线那项和下一节的第一条，会把这套“成功”的含金量说清楚。

四、被数据打脸的三次

第一次：自己造的数据全过，真实数据 1/14

合成数据里的异常，是“我知道它在哪里，然后把它放进去”。真实数据里的灾害事件，是“气象部门按业务标准认定的一段天气过程”。

这两件事根本不是同一个任务。

一次热浪警报，在统计意义上可能只是相对气候态 $Z \approx 1.4$ 的偏移——它不突出，但它确实是一次灾害。反过来，统计上最突出的那 20 个点，可能只是传感器抖动。

所以门禁的 0.9667 只在“注入型异常”上成立；真实事件召回 1/14 才是对外的诚实答案。

有意思的是：加上一条独立的“绝对极端”通道（按经验 p99/p1 取尾，与统计异常分开）之后，14 条事件里能覆盖 7 条——**一半的事件不是“统计异常”，而是“绝对值极端”**。这说明问题不在阈值调得松紧，而在把两种不同的东西塞进了同一个指标。

第二次：Agent 并不比流水线聪明

这是最反直觉的一次。

13 个天气用例，同一份数据、同一个问题，跑两遍：一遍固定流水线（12 项计划全跑），一遍让模型自己决定先调哪个工具。

维度	流水线	Agent
平均步数	5.00	2.54
平均结论数	39.00	29.92（−23.3%）
需求覆盖率	1.00	1.00
规则计划覆盖率	1.00	0.66
墙钟（均值）	12.09 s	12.77 s（+5.6%）
token	0	53,924

Agent 少跑了三分之一的分析项，结论少了 23.3%，还更慢——因为调用模型本身要花时间。

那这 118 条差异是“模型判断更精炼”吗？把每条结论按类别对齐后：**118 条差异 100% 归因于“工具没被执行”（`tool_not_run`），归因于“检测口径不同”的是 0 条。**

也就是说，它不是做得更准，是做得更少。两边的需求覆盖率都是 1.00，说明用户问的那件事都答了；差别在“顺带做的检查”。

这个结果直接改了我对这个功能的产品定位：Agent 不是“更聪明的分析师”，它是“可审计的执行器”——每一步都能查、能重放、能在预算内停下来。把它包装成前者，第一次对比实验就会打脸。

第三次：用户先撞上的是口径错，不是算法错

真实数据上暴露的两个最严重缺陷，都和统计方法无关：

1. **单位误判**：某地 5—6 月气温上升，被规则判成“华氏度段”，5,977 行按 `(x-32)/1.8` 改写。数据没错、算法没错，是“季节性升温”与“单位跳变”在形状上太像。

   修法不是调阈值，而是加一个更强的判据：**阶跃之后必须“换算后边界连续”**，单侧转移不自动换算。修完零误换算，8 月的峰值月回到 8 月。

2. **多站点判重**：三站点长表里，几个站点共享同一条时间轴，质量报告按整表 `timestamp` 统计，报出 17,568 条“重复”。清洗阶段本身是对的（按站点透视，8,784 行 × 15 列），错的只是报告口径。

   修法是把判重键从 `timestamp` 改成 `(station, timestamp)`。修完 17,568 → 0。

这两个 bug 有个共同点：**它们都会让用户对数据本身产生错误认知**，而且都在“算法看起来没问题”的地方。单位误判会让后面所有温度结论作废，重复误报会让用户以为数据采集坏了。

所以后来加了一条规则：每一条质量问题都必须生成一条“数据口径”结论。80 次真实数据运行里，这个问题从 2/20 有覆盖变成 20/20，需求覆盖率 0.800 → 1.000。

五、把成本说成人话

Agent 模式的成本是 token，但用户关心的是钱。

所以价目表必须由使用者自己给：单位 token 价格、来源说明，缺一项就报 `PRICE_TABLE_INVALID`，**绝不按 0 计价**——那等于假装免费。金额一律标“估算”，并且预算在调用之前预扣，而不是调用完了再算账。

没有金额语义的预算，对使用者没有意义；“花了 8,000 token”这句话，除非他知道这是什么价钱。

六、工程侧我只提一句

为了让上面这些结论能被别人复算，项目最终是：53 个测试文件、数百项测试用例、Python 3.12 与 3.13 双版本 CI、Docker 镜像真跑一次分析、`pip install mtsa-agent` 之后 `mtsa demo` 一条命令出报告。

这些不是“技术栈展示”，是“我的结论你可以自己验一遍”的最低成本。

七、写在最后

回到开头那两个数字。

24/24 证明的是：在我定义的任务上，系统符合我的预期。 0.0714 证明的是：在现实定义的任务上，系统还差得很远。

两个都要写进报告，缺一个，这份报告就变成宣传材料。

我更愿意把这套实践叫**可审计性**：不是让系统“什么都会”，而是让它能回答“这条结论从哪来、哪些没做、为什么没做、花了多少钱”。这件事的价值不在演示时，在出问题时——用户第一次质疑你的结论时，你能不能在三分钟内指出它的出处。

下一步是真实事件召回（多尺度残差、持续段与漂移）。但我给自己定了个前置条件：**至少有一个真实外部用户反馈过“这个结果不对”**。没有反馈的召回率优化，是在猜用户要什么。

最后一个建议给做类似系统的同行：如果你的评估全绿，先怀疑评估，而不是庆祝。

附：文中数字的出处

数字	出处
门禁 24/24、事件召回 0.9667、变化点 F1 0.50	`evals/conclusion_gate_v2_results.json`
真实数据 80/80、证据覆盖 1.000	`evals/real_data_results.json`
事件级召回 0.0714（1/14）、极端通道 7/14	`evals/real_data_results.json`（R13 / R14）
Agent 对照 39.00 → 29.92、+5.6% 墙钟、53,924 token	`evals/mode_compare_results.json`
差异 118 条 100% 归因 `tool_not_run`	`evals/conclusion_diff_results.json`、`doc/评估设计.md` 6.x
单位误判 5,977 行、重复误报 17,568 条	`doc/评估设计.md`、`spec/archive/项目承接已完成.md` 第 34–35 章
需求覆盖 0.800 → 1.000	`spec/archive/项目承接已完成.md` 第 36 章
53 个测试文件、数百项用例	`evals/test_logs/summary.json`